

AgroVision AI — Complete Project Report

Project Title: AgroVision AI — AI-Powered Plant Disease Detection and Farmer Assistance Platform

Mentor : Dr. Akshay Pandey

Authors: Aashrith, Tharun, Karan, Kishore, Ruthwik

Affiliation: IIITDM Jabalpur

Table of Contents

- 1. [Executive Summary](#)
 - 2. [Initial Goal and Motivation](#)
 - 3. [Project Phases Overview](#)
 - 4. [Phase 1 — Research and Model Training](#)
 - 5. [Phase 2 — AgroVision AI Application](#)
 - 6. [System Architecture](#)
 - 7. [Current Features](#)
 - 8. [Tech Stack](#)
 - 9. [Deployment](#)
 - 10. [Limitations and Disclaimers](#)
 - 11. [Future Plans](#)
 - 12. [Conclusion](#)
 - 13. [References](#)
-

1. Executive Summary

AgroVision AI is an end-to-end farmer assistance platform that identifies plant leaf diseases from smartphone photos and delivers practical, actionable guidance. The project evolved in two phases:

- **Phase 1 (Research):** A controlled comparative study of five ImageNet-pretrained CNN architectures on a 36-class plant disease dataset. ResNet50 achieved the highest validation accuracy (99.04%), while EfficientNetB0 offered the best accuracy-vs-training-time tradeoff.
- **Phase 2 (Application):** A production-ready full-stack web application (Next.js + FastAPI) that deploys trained models, generates AI-powered farmer guidance, and provides weather risk, scan history, progress tracking, multilingual support, voice, and WhatsApp sharing.

The goal throughout has been to bridge academic research with real-world usability for farmers.

2. Initial Goal and Motivation

2.1 Problem

Crop losses from biotic stresses (fungi, bacteria, viruses) and abiotic stresses (nutrient deficiencies, drought) often appear as visible symptoms on leaves — spots, chlorosis, necrosis, powdery growth, and more. Expert

diagnosis is reliable but does not scale to large farms or frequent monitoring. Computer vision offers a complementary tool: a classifier can suggest likely categories from a smartphone photo, enabling triage before expert review.

2.2 Project objectives

- 1. **Research:** Fairly compare CNN backbones for multi-class plant disease classification and select the best model for deployment.
- 2. **Application:** Build a farmer-friendly web platform that goes beyond classification — providing treatment, prevention, voice help, and follow-up tools.
- 3. **Impact:** Make disease detection accessible, understandable, and actionable for farmers in India.

2.3 Long-term vision

Phase 1 was designed as the foundation for **ensemble learning** — combining multiple trained models for a more robust predictor than any single backbone. The full AgroVision AI app represents the deployment and productization of this research.

3. Project Phases Overview

Phase	Focus	Key deliverables
Phase 1	Comparative CNN study	Trained models, metrics, Streamlit demo, Grad-CAM tooling
Phase 2	Full-stack farmer app	AgroVision AI (Next.js + FastAPI), medicinal model, AI guidance, deployment

Phase 1: Research

Dataset → Train 5 CNNs → Compare metrics → Select model → Streamlit prototype

↓

Phase 2: Application

Deploy EfficientNetB0 → Build full web app → AI guidance → Farmer features → Vercel + server

4. Phase 1 — Research and Model Training

4.1 Research title

Comparative Study of Pretrained Convolutional Neural Networks for Multi-Class Plant Leaf Disease Classification

4.2 Dataset

Property	Value
Dataset name	Plant_leave_diseases_dataset_with_augmentation

Property	Value
Task	Multi-class classification into 36 disease/health/background categories
Style	PlantVillage-style (folder-per-class)
Label source	<code>class_names.json</code> (alphabetical folder order)
Image size	224 × 224 RGB
Train/validation split	80/20, seed = 123
Batch size	24

Notable class: `Background_without_leaves` — helps the model reject non-leaf regions, useful for real deployment.

Supported crops (36 classes): Apple, Blueberry, Cherry, Corn, Grape, Orange, Peach, Pepper, Potato, Raspberry, Soybean, Squash, Strawberry, Tomato (plus healthy and background categories).

4.3 Models compared

Five ImageNet-pretrained CNN architectures were evaluated under identical conditions:

Model	Family	Key characteristic
MobileNetV2	Efficient mobile	Smallest footprint (~2.6M trainable params)
ResNet50	Residual networks	Strong default for transfer learning
VGG16	Deep conv stacks	Simple but computationally expensive
EfficientNetB0	Compound scaling	Strong accuracy vs efficiency tradeoff
DenseNet121	Dense connectivity	Parameter-efficient feature reuse

4.4 Training protocol (fair comparison)

All models were trained under a **standardized experimental protocol** to ensure reproducibility:

- 1. Remove ImageNet classification head (`include_top=False`)
- 2. **Freeze** all backbone weights (`base_model.trainable = False`)
- 3. Apply backbone-specific `preprocess_input` normalization
- 4. Attach a **uniform classification head**:
 - GlobalAveragePooling2D
 - Dense(256) + ReLU
 - Dropout(0.5)
 - Dense(36) + Softmax

Training hyperparameters:

Parameter	Value
Optimizer	Adam

Parameter	Value
Learning rate	1×10^{-4}
Epochs	10
Loss	Sparse categorical cross-entropy
Train/val split	80/20, seed 123

4.5 Evaluation metrics

To handle class imbalance fairly, we reported:

- Accuracy (overall correctness)
- Weighted precision / recall / F1 (by class frequency)
- **Macro** precision / recall / F1 (unweighted — important under imbalance)
- Balanced accuracy (mean of per-class recall)
- **MCC** (Matthews Correlation Coefficient)
- Confusion matrices for qualitative error analysis
- Training wall-clock time and parameter counts

4.6 Experimental results

Table 1 — Validation metrics (frozen backbone; 10 epochs; seed 123)

Model	Val Accuracy	Balanced Acc	F1 (weighted)	F1 (macro)	P (macro)	R (macro)	MCC	Train (min)
ResNet50	0.9904	0.9883	0.9904	0.9886	0.9889	0.9883	0.9900	27.49
EfficientNetB0	0.9855	0.9826	0.9854	0.9830	0.9836	0.9826	0.9849	13.17
DenseNet121	0.9794	0.9749	0.9793	0.9756	0.9768	0.9749	0.9786	25.18
MobileNetV2	0.9747	0.9700	0.9747	0.9706	0.9714	0.9700	0.9737	13.02
VGG16	0.9710	0.9651	0.9708	0.9656	0.9666	0.9651	0.9698	64.01

Ranking: ResNet50 ranked first across all reported accuracy-oriented metrics.

Figures — Phase 1 research (place images here)

Save notebook outputs into *report-assets/* using the filenames below.

Figure 4.1 — Model comparison bar chart

Bar chart comparing validation accuracy, F1, and balanced accuracy across all five CNNs.

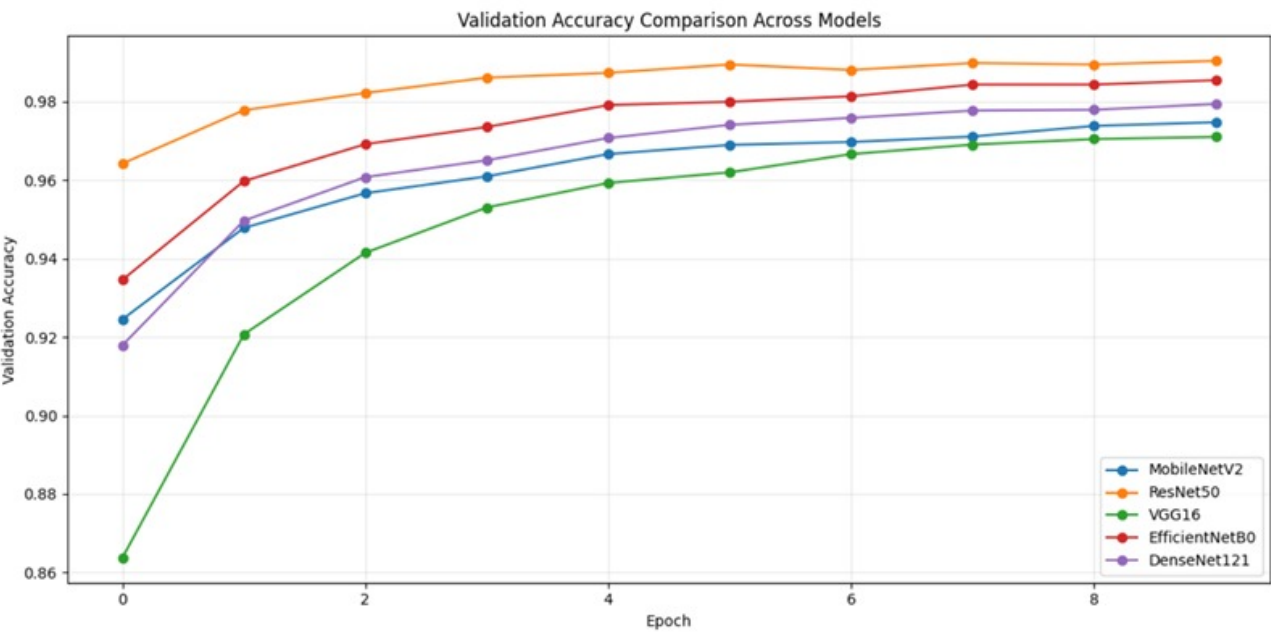


Figure 4.2 — validation loss vs epochs
Scatter or bar plot showing the accuracy–efficiency tradeoff (why EfficientNetB0 was chosen for deployment).

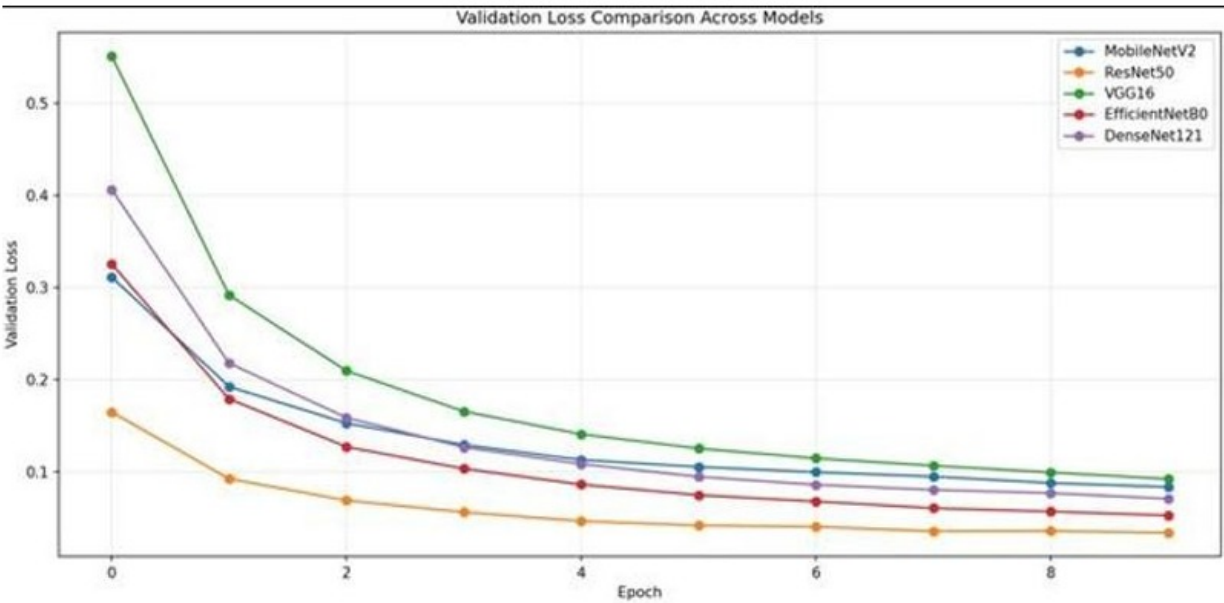


Figure 4.3 — Training time and validation curves
Training/validation accuracy and loss curves per model (or overlaid validation accuracy).

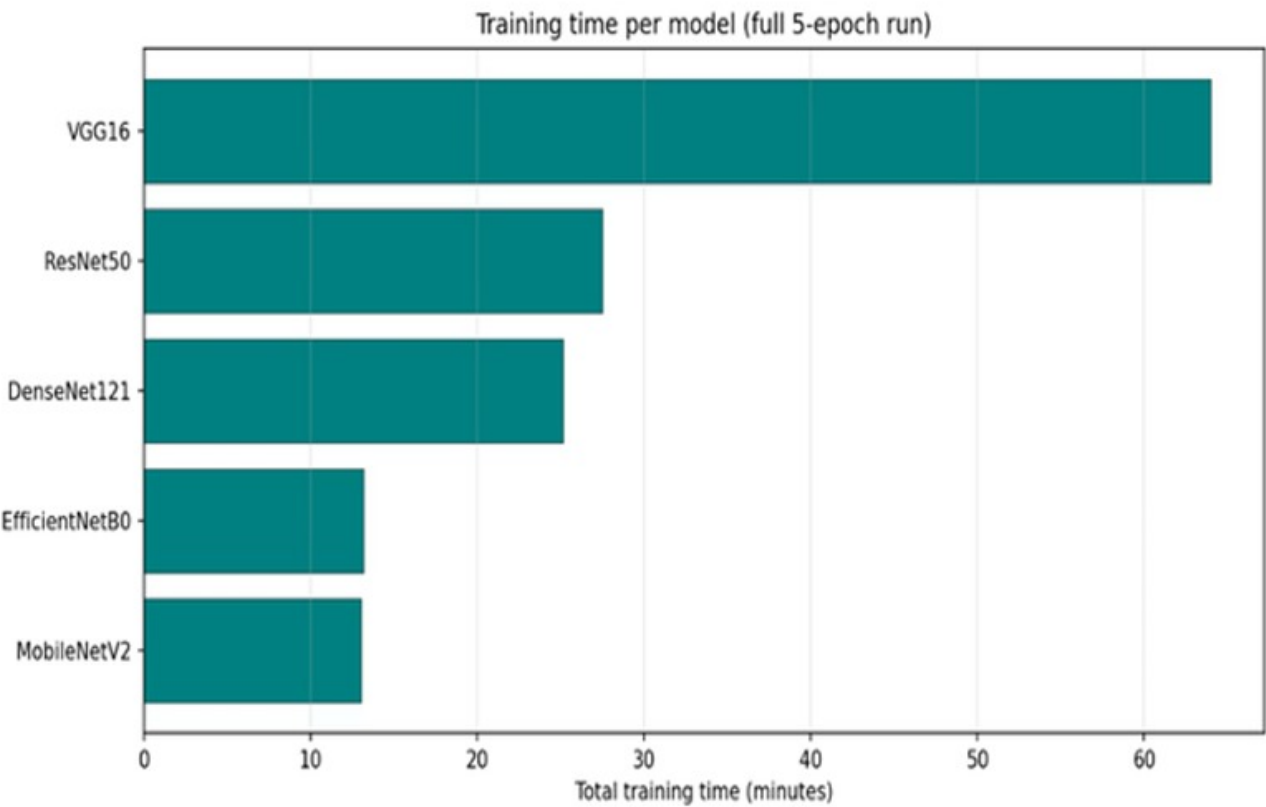


Figure 4.4 — Confusion matrix (ResNet50)

Confusion matrix for the best-performing model (ResNet50). Optionally add EfficientNetB0 as a second image.

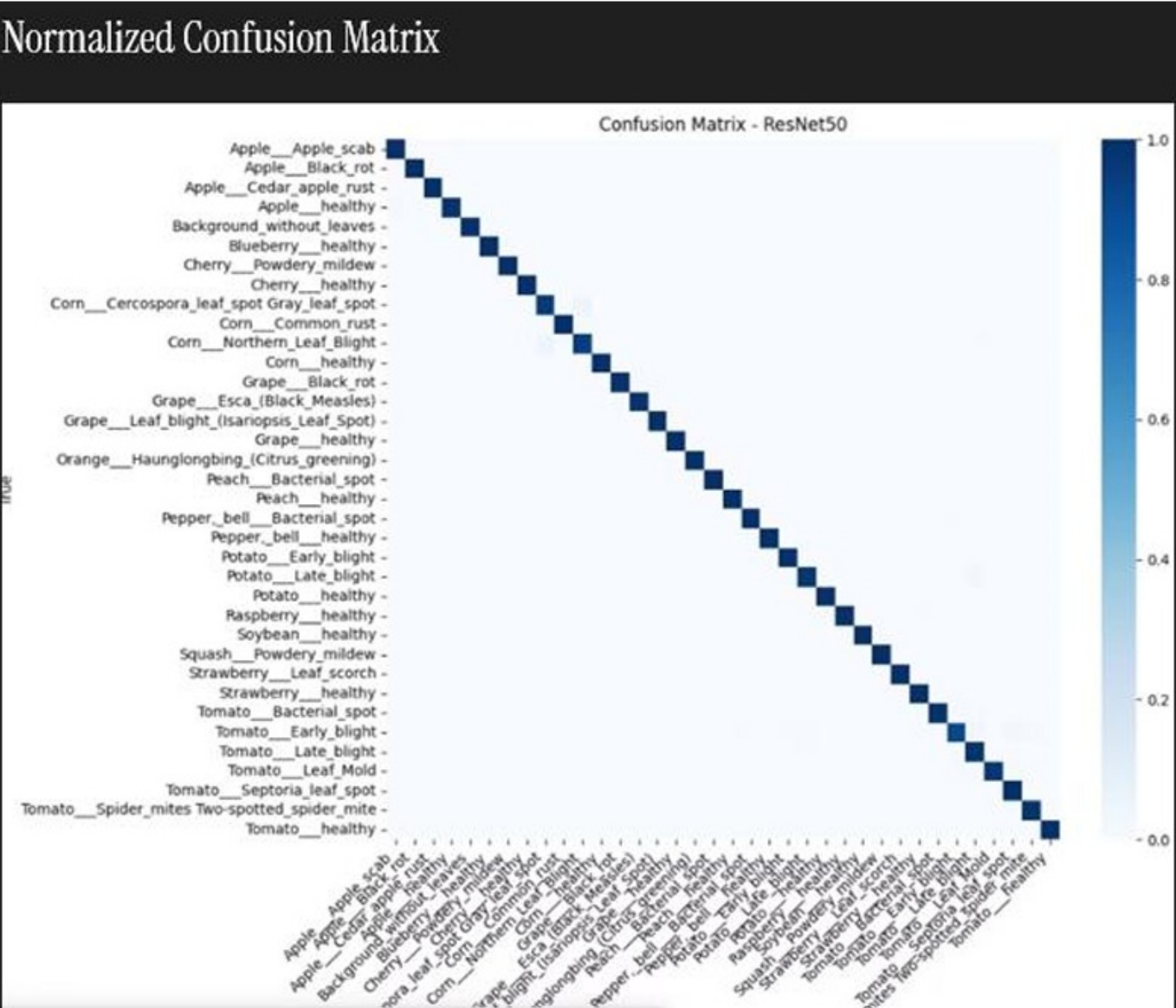


Table 2 — Model size and training cost

Model	Parameters	Train (min)
MobileNetV2	2,595,172	13.02
EfficientNetB0	4,386,759	13.17
DenseNet121	7,309,156	25.18
ResNet50	24,121,508	27.49
VGG16	14,855,268	64.01

4.7 Key findings from Phase 1

- 1. **All five models exceeded 97% validation accuracy** under the shared protocol — frozen ImageNet features plus the common head are well matched to this dataset.
- 2. **ResNet50** delivered the best single-model performance (99.04% accuracy, macro-F1 0.9886, MCC 0.9900).

- 3. **EfficientNetB0** was the strongest second model with training time comparable to MobileNetV2 (~13 min) while outperforming it on accuracy — best accuracy-vs-efficiency tradeoff.
- 4. **MobileNetV2** remained the smallest model (~2.6M parameters) with competitive ~97.5% accuracy — suitable for resource-constrained edge deployment.
- 5. **VGG16** trained slowest (64 min) despite fewer parameters than ResNet50, reflecting architectural compute differences.
- 6. ResNet50's advantage was most visible on **macro-averaged metrics**, important when class frequencies differ.

4.8 Model selection for deployment

Although ResNet50 achieved the highest accuracy, **EfficientNetB0 was selected for the AgroVision AI production app** because:

- Near-top accuracy (98.55% vs 99.04%)
- **~2× faster training** (13 min vs 27 min)
- **~5.5× fewer parameters** (4.4M vs 24.1M) — faster inference and lower memory on server
- Natural candidate for Phase 2 ensemble pairing with ResNet50

4.9 Phase 1 engineering artifacts

Artifact	Purpose
<code>Comparative_Study_CNN_refactored.ipynb</code>	Training and comparison notebook
<code>comparison_results.csv</code>	Exported metrics for all models
<code>leaf_disease_app.py</code>	Streamlit demo for real-time prediction
<code>gradcam_saved_weights.py</code>	Grad-CAM heatmaps for model interpretability
Trained <code>.h5</code> model files	Exported weights for deployment

4.10 Explainability (Grad-CAM)

Grad-CAM (Gradient-weighted Class Activation Mapping) was integrated to highlight image regions that influence model predictions. This bridges the gap between research accuracy and farmer trust. Grad-CAM is a diagnostic sensitivity tool — it highlights what the model attends to, not causal plant pathology.

Figure 4.5 — Grad-CAM on correct prediction
Heatmap overlay showing which leaf regions the model used for a correct disease classification.

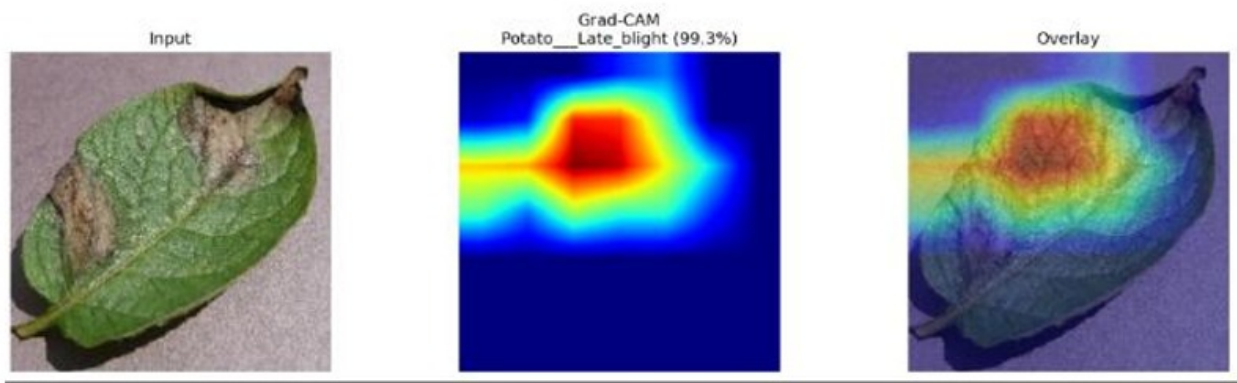


Figure 4.7 — Phase 1 Streamlit prototype (optional)
Early `leaf_disease_app.py` demo used before building the full web app.

4.11 Medicinal plant model (extension)

Beyond the 36-class crop model, a separate **medicinal plant disease model** was trained:

- **Plants:** Camphor, HariTaki, Neem, Sojina
- **Classes:** 13 (disease + healthy per plant)
- **Notebook:** `AgroVision_Medicinal_Plant_Training.ipynb`
- **Backbone:** EfficientNetV2B0 (recommended)
- Integrated into the app as a separate "Medicinal" prediction mode

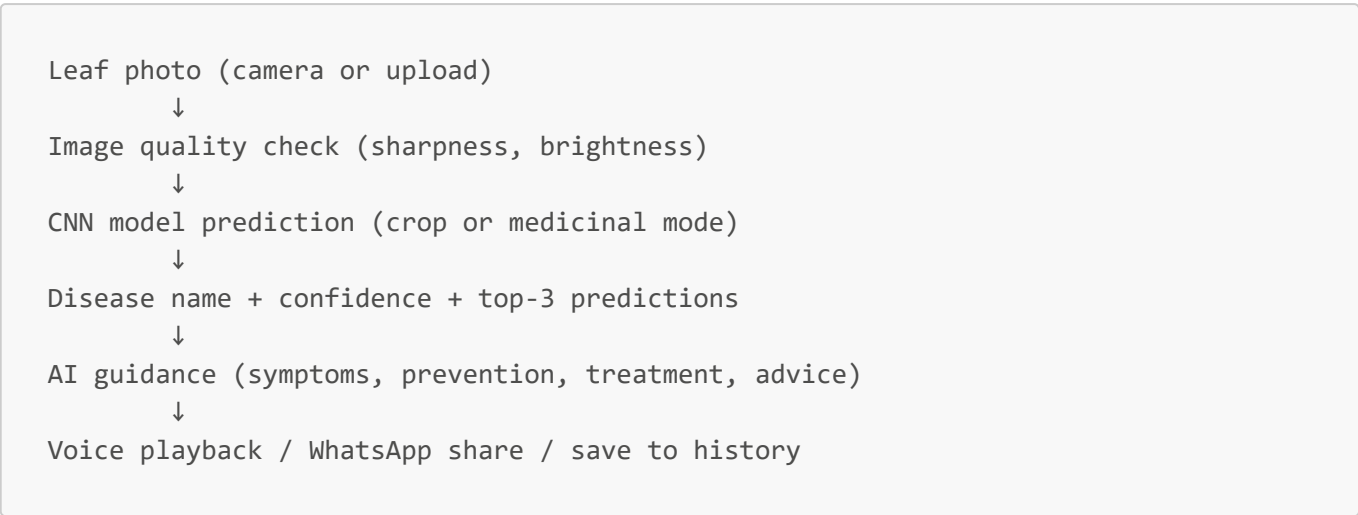
5. Phase 2 — AgroVision AI Application

After completing the research phase, the team built a full production web application that goes far beyond the Streamlit prototype.

5.1 What changed from Phase 1 to Phase 2

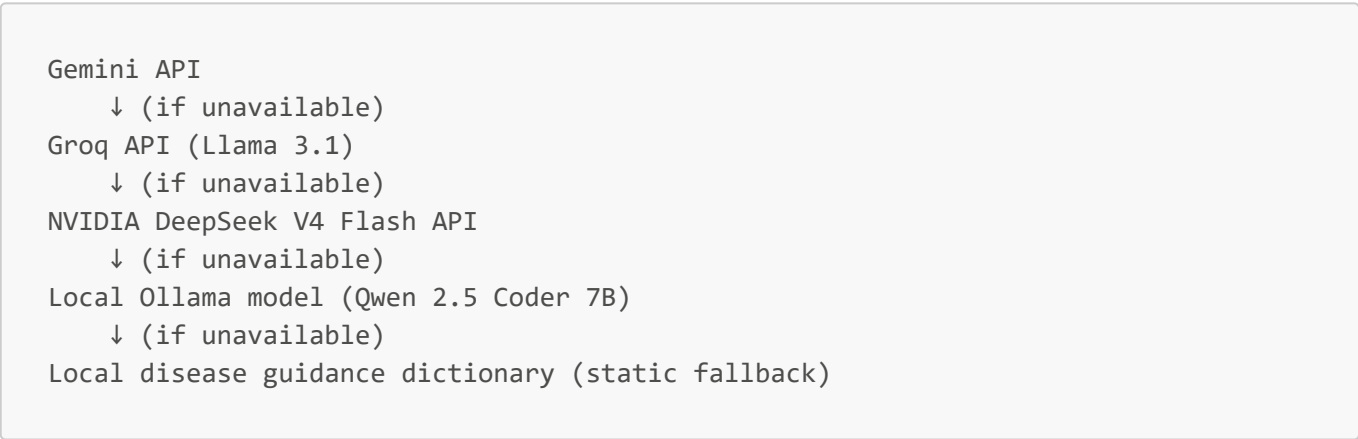
Aspect	Phase 1 (Research)	Phase 2 (AgroVision AI)
Interface	Streamlit demo	Next.js responsive web app
Guidance	Prediction only	AI-generated symptoms, treatment, prevention
Languages	English only	English, Hindi, Telugu
Voice	None	Text-to-speech explanations
Sharing	None	WhatsApp integration
Weather	None	Local weather risk analysis
History	None	Scan history, reminders, progress tracker
Models	Single crop model	Crop + medicinal dual models
Deployment	Local script	Vercel + college server / Render

5.2 Diagnosis workflow



5.3 AI guidance system

After CNN prediction, farmer-friendly text is generated using a **multi-provider fallback chain**:



This ensures the app continues working even when online AI providers are unavailable.

5.4 Application pages

Page	Description
Home / Diagnosis	Upload or capture leaf image, run prediction, view results
Results Panel	Disease, confidence, top-3, AI guidance, severity, voice, WhatsApp
Disease Library	Searchable reference for all 36 trained crop classes
Disease Detail	Per-disease information, symptoms, field checklists
Scan History	Past diagnoses stored locally on device
Disease Progress Tracker	Tracks whether a plant's condition is improving, stable, or worsening over time
Follow-up Reminders	Recheck tasks after treatment
Weather Risk	Humidity, rain, temperature, wind → disease spread risk

Page	Description
Admin Dashboard	Backend health, model info, API diagnostics

5.5 Disease Progress Tracker (detailed)

The **Disease Progress Tracker** is a dedicated feature (</progress>) that helps farmers monitor whether a plant's disease condition is **getting better, staying the same, or getting worse** after treatment.

How it works:

- Farmer runs an initial diagnosis on the home page — the scan is saved automatically in **Scan History**.
- After a few days of treatment, the farmer opens the **Progress Tracker** page and selects the earlier scan.
- They upload a **new leaf photo** of the same plant.
- The app runs CNN prediction on the new image and **compares it with the previous scan**.
- The system assigns one of four statuses:

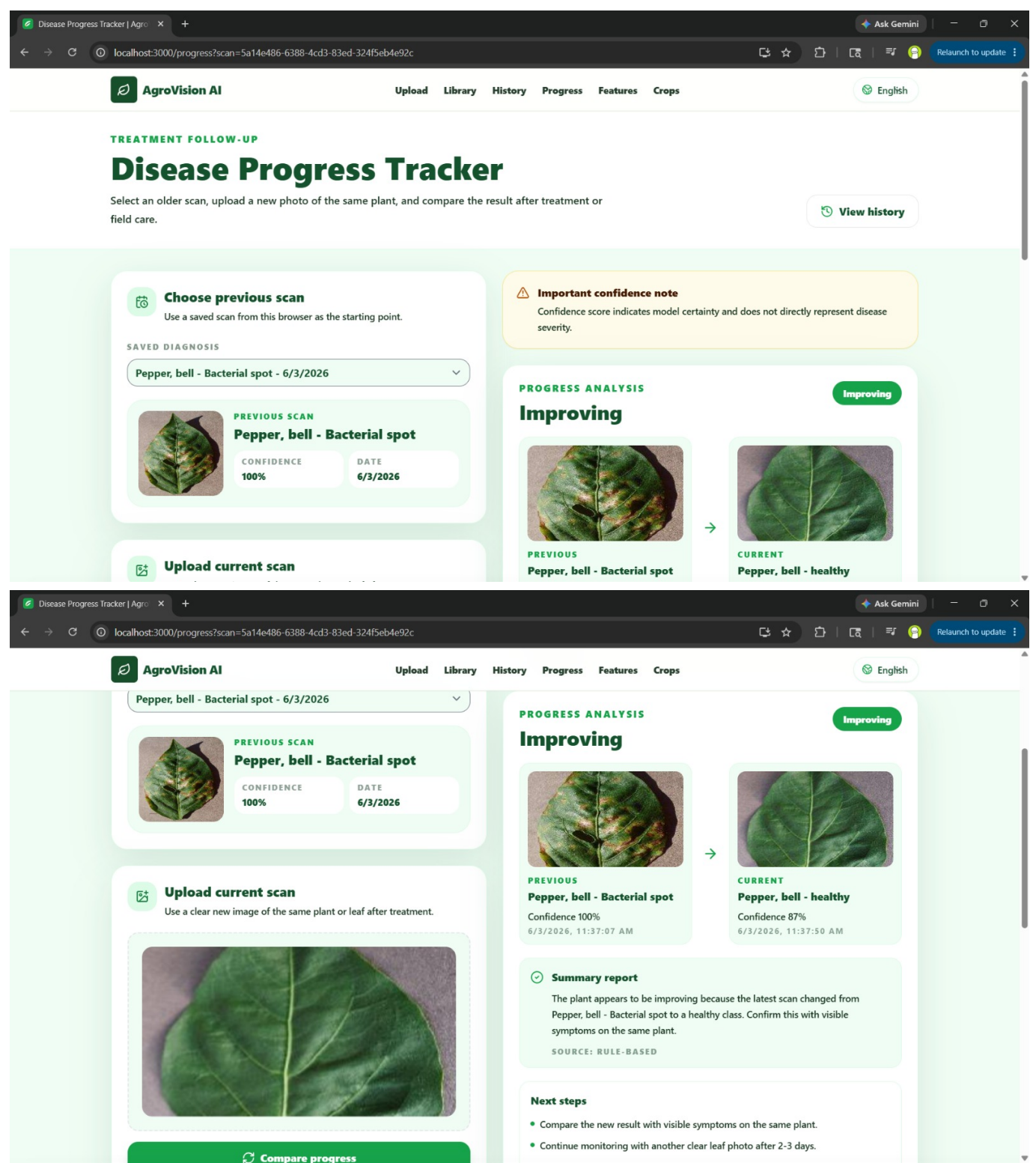
Status	Meaning	Example
Improving	Disease appears to be decreasing	Previous scan: Tomato Early Blight → Current scan: Healthy
Stable	Same disease still detected; no clear change	Both scans: Tomato Early Blight
Worsening	Disease appears to be increasing	Previous scan: Healthy → Current scan: Tomato Late Blight
Inconclusive	Not enough reliable evidence	Different crops, low confidence, or disease label changed unexpectedly

- The backend may also generate an **AI progress report** with a plain-language summary and next steps (continue treatment, recheck in 2–3 days, contact an expert if worsening).

Important note: Confidence score reflects **model certainty**, not disease severity. The tracker compares disease **labels** and crop type — it does not claim severity increased or decreased based on confidence alone.

Figure 5.1 — Disease Progress Tracker

Screenshot showing previous vs current scan side-by-side with Improving / Stable / Worsening status.



5.6 Figures — Phase 2 application (place app screenshots here)

Take these from the live app or local dev server. Save into *report-assets/* with the filenames below.

Figure 5.2 — Home / diagnosis page

Upload area, camera capture, crop vs medicinal mode selector.

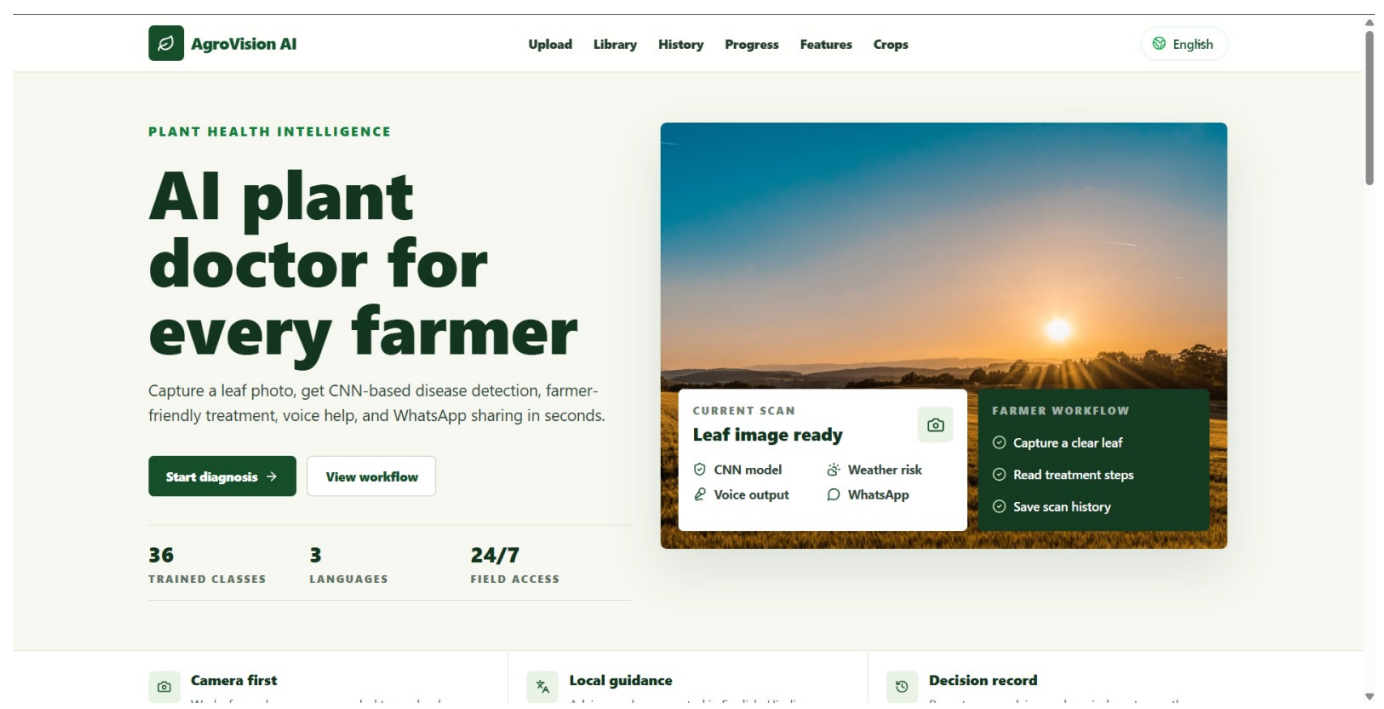


Figure 5.3 — Prediction results and AI guidance
Disease name, confidence, top-3 predictions, symptoms, prevention, treatment.

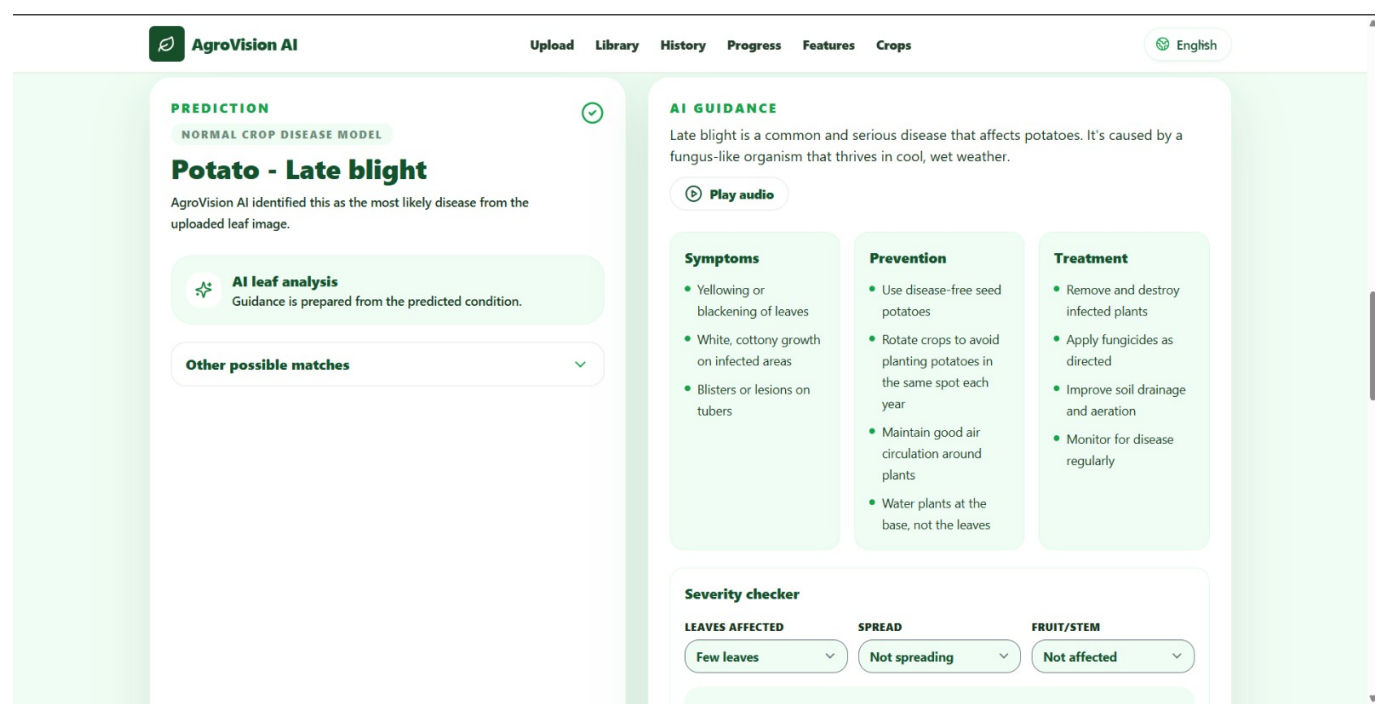


Figure 5.4 — Disease library
Searchable list of all 36 trained crop disease classes.

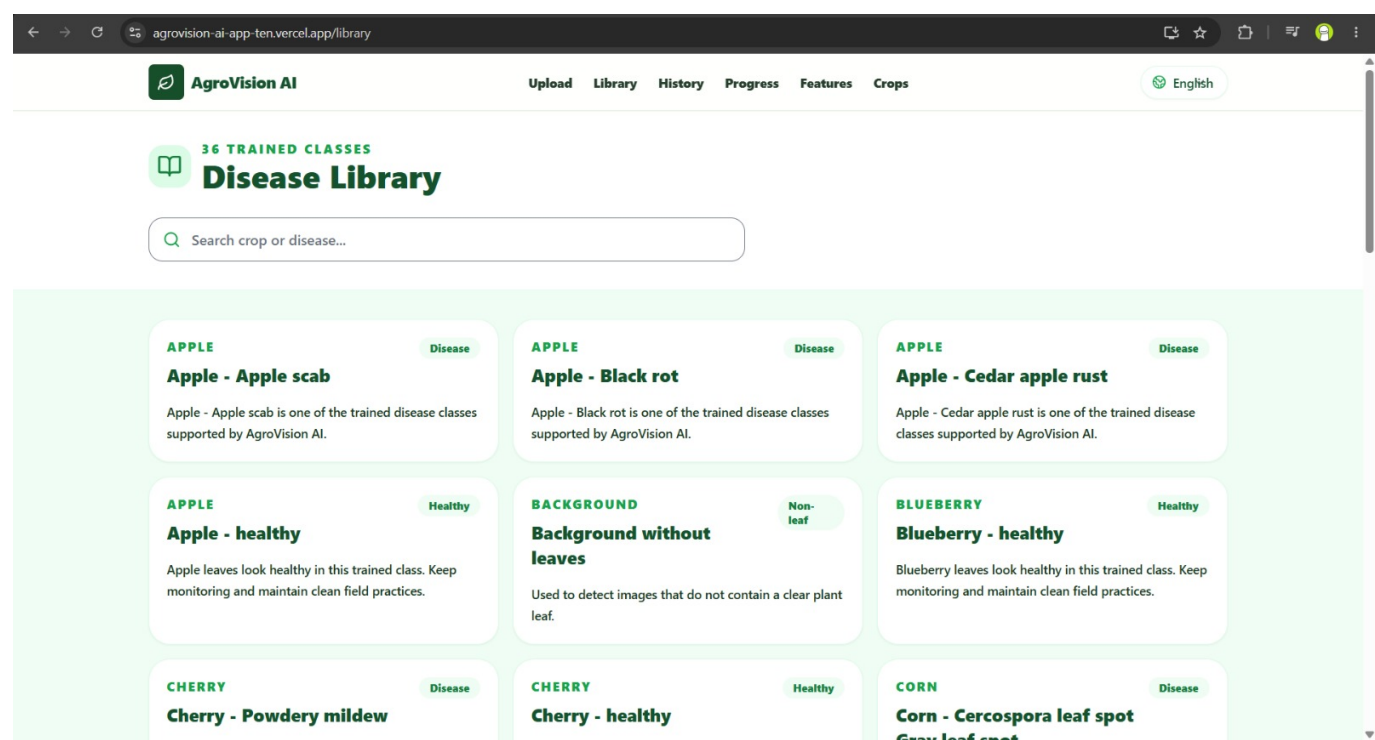


Figure 5.5 — Scan history
Past diagnoses with thumbnails and follow-up links.

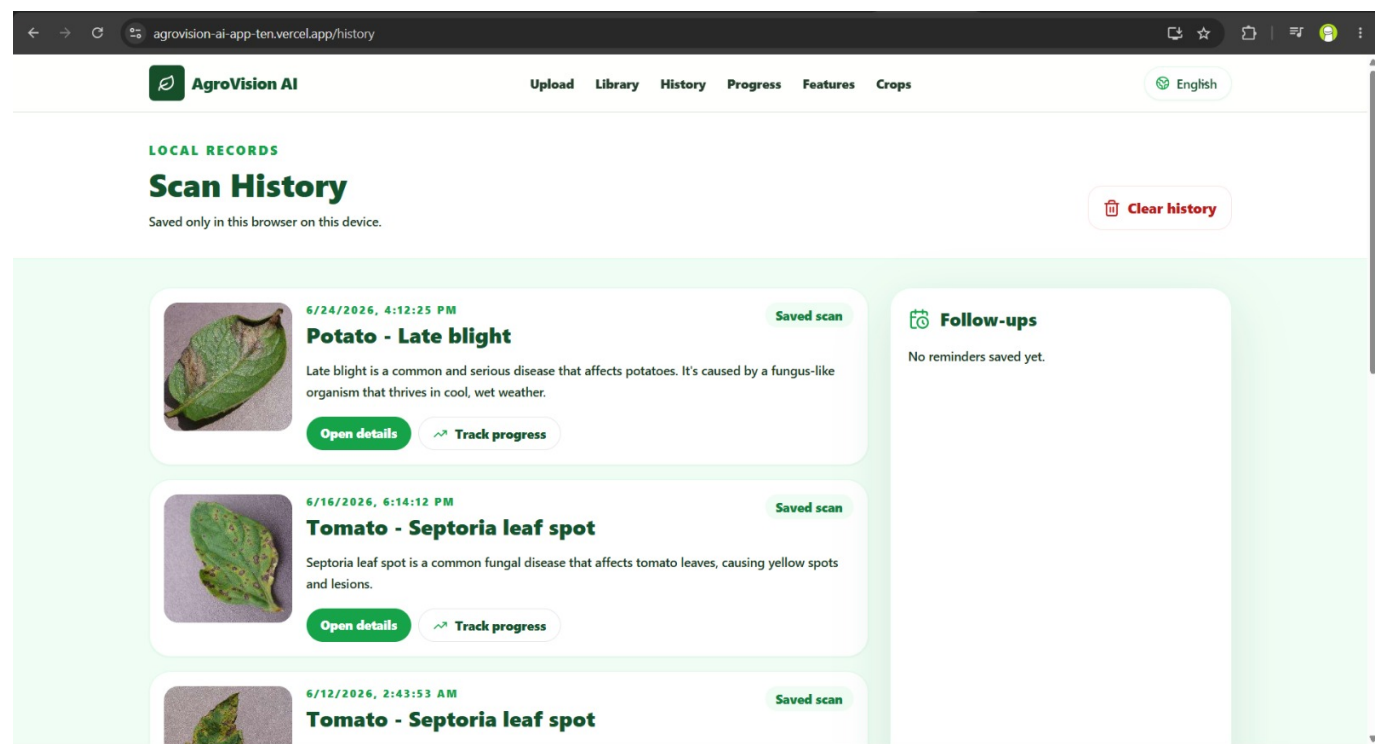
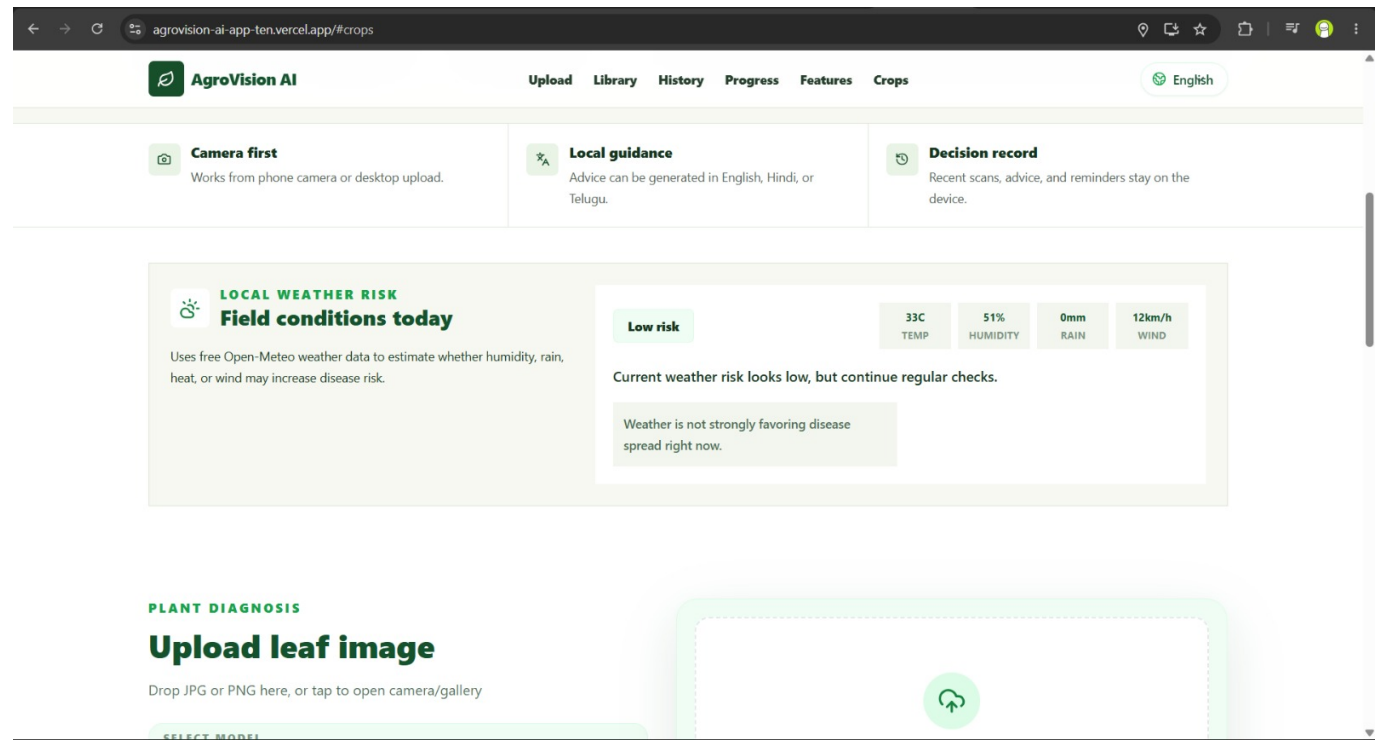
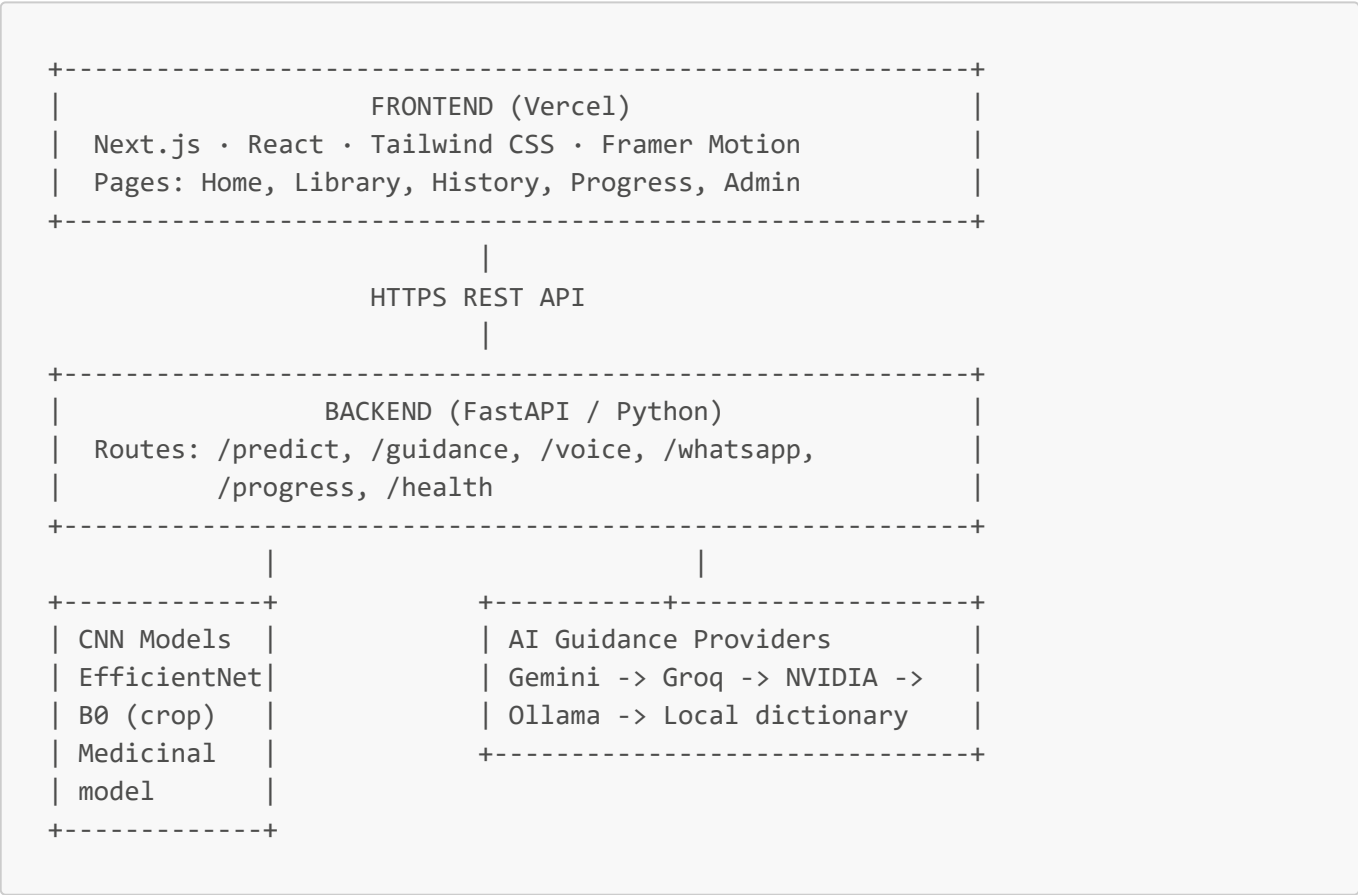


Figure 5.6 — Weather risk panel
Local humidity, rain, temperature, and disease-spread risk level.



6. System Architecture



7. Current Features

7.1 Core diagnosis

- Leaf image upload and mobile camera capture (JPG/PNG, max 8 MB)

- Dual model modes: **Crop** (36 classes) and **Medicinal** (13 classes)
- Top-3 predictions with confidence scores
- Low-confidence warnings when plant is outside trained dataset

7.2 Farmer guidance

- AI-generated symptoms, prevention, treatment, and plain-language advice
- Multi-provider fallback for reliability
- Voice explanation (ElevenLabs / gTTS)
- WhatsApp result sharing (Twilio / wa.me fallback)

7.3 Field support tools

- **Weather risk analysis** — Open-Meteo API with geolocation (humidity, rain, wind, temperature)
- **Severity checker** — observation-based questionnaire (affected leaves, spread, fruit/stem)
- **Disease Progress Tracker** — compares a new leaf scan with a previous scan to show whether the plant's condition is **improving** (disease decreasing), **stable** (unchanged), or **worsening** (disease increasing); includes AI-generated next steps
- **Follow-up reminders** — recheck tasks after treatment
- **Scan history** — local device storage with thumbnails; links directly to Progress Tracker

7.4 Knowledge base

- Searchable disease library (36 crop classes)
- Detailed per-disease information pages

7.5 Accessibility

- **Multilingual UI:** English, Hindi, Telugu
- Mobile-first responsive design
- Image quality tips before prediction

7.6 Admin

- Backend health monitoring
- Model file status and class counts
- API call diagnostics (token-protected)

8. Tech Stack

Layer	Technologies
Frontend	Next.js, React, TypeScript, Tailwind CSS, Framer Motion
Backend	FastAPI, Python 3.11, TensorFlow/Keras
ML (crop)	EfficientNetB0, 36-class .h5 model
ML (medicinal)	EfficientNet-based, 13-class .h5 model

Layer	Technologies
AI/LLM	Gemini, Groq, NVIDIA DeepSeek, Ollama (Qwen)
Voice	ElevenLabs, gTTS
Messaging	Twilio WhatsApp, wa.me fallback
Weather	Open-Meteo API
Explainability	Grad-CAM (Phase 1)
Deployment	Vercel (frontend), Render / college server (backend)

9. Deployment

Current setup

- **Frontend:** Vercel (agro-vision-ai-ochre.vercel.app)
- **Backend:** FastAPI on Render or college Linux server
- **Demo mode:** Backend on laptop + ngrok tunnel

College server deployment

- Python 3.11, virtual environment, systemd service
- Model files on server ([EfficientNetB0.h5](#), medicinal model, JSON class files)
- Optional Ollama/Qwen for offline AI guidance
- Nginx reverse proxy, CORS restricted to Vercel frontend

10. Limitations and Disclaimers

Research limitations (Phase 1)

- Validation split only — not a fully independent field test
- Controlled backgrounds in PlantVillage-style data; models may learn shortcuts (background, lighting)
- Frozen backbone limits adaptation; fine-tuning may change model rankings
- Grad-CAM highlights sensitivity, not causal pathology

Application limitations (Phase 2)

- Predictions depend on trained classes only — unknown plants may be misclassified
- Confidence reflects model certainty, **not** field severity
- Scan history and reminders are device-local — not synced across phones
- AI guidance is informational — farmers should verify with local experts before spraying
- Medicinal model covers 4 plant types only
- Backend needs sufficient RAM for TensorFlow + optional Ollama (16 GB recommended)

11. Future Plans

From Phase 1 research roadmap

1. **Ensemble learning** — soft voting / probability averaging across ResNet50 + EfficientNetB0
2. **Stacking** — meta-classifier on concatenated logits
3. **Diversity-aware ensembles** — different architectures to reduce correlated errors
4. **Calibration** — temperature scaling for better probability semantics
5. **Held-out test set** — field images from real farm conditions

Application roadmap

1. Expand dataset with **Indian regional crops** (rice, wheat, cotton, chilli, mango)
2. Improve medicinal plant coverage
3. **Offline PWA** support for low-connectivity areas
4. **User accounts and cloud sync** for scan history
5. **Push notifications** for follow-up reminders
6. **Severity from image** — segmentation / affected area detection
7. **Mobile native app** (Android) with on-device inference (TensorFlow Lite)
8. **Regional language expansion** — Tamil, Kannada, Marathi, Bengali
9. **Community disease map** — anonymous regional outbreak tracking
10. Integration with **Krishi Vigyan Kendra (KVK)** networks

12. Conclusion

AgroVision AI represents a complete journey from academic research to a deployable farmer assistance platform:

Phase 1 established that transfer learning with frozen ImageNet backbones achieves >97% validation accuracy on 36-class plant disease classification. ResNet50 led on accuracy (99.04%), while EfficientNetB0 offered the best accuracy-efficiency tradeoff for deployment.

Phase 2 transformed this research into a production web application with AI guidance, multilingual support, weather context, voice, WhatsApp sharing, progress tracking, and a disease library — features that address real farmer needs beyond simple classification.

The project demonstrates practical skills across the full ML pipeline — dataset preparation, model training, comparative evaluation, explainability, API design, full-stack web development, AI service integration, and cloud deployment — while targeting a meaningful problem in Indian agriculture.

The next step is **ensemble learning** (combining ResNet50 + EfficientNetB0) and evaluation on held-out field images to further improve robustness before wider deployment.

13. References

1. Hughes, D. P., & Salathé, M. (2015). *An open access repository of images on plant health to enable the development of mobile disease diagnostics*. arXiv:1511.08060.
2. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using Deep Learning for Image-Based Plant Disease Detection. *Frontiers in Plant Science*, 7:1419.

3. Sladojevic, S., et al. (2016). Deep Neural Networks Based Recognition of Plant Diseases by Leaf Image Classification. *Computational Intelligence and Neuroscience*.

4. Ferentinos, K. P. (2018). Deep learning models for plant disease detection and diagnosis. *Computers and Electronics in Agriculture*, 145, 311–318.

5. Too, E. C., et al. (2019). A comparative study of fine-tuning deep learning models for plant disease identification. *Computers and Electronics in Agriculture*, 161, 272–279.

6. Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *ICCV*.

7. Simonyan, K., & Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. *ICLR*.

8. He, K., et al. (2016). Deep Residual Learning for Image Recognition. *CVPR*.

9. Huang, G., et al. (2017). Densely Connected Convolutional Networks. *CVPR*.

10. Sandler, M., et al. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *CVPR*.

11. Tan, M., & Le, Q. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *ICML*.

Appendix A — Reproducibility Checklist (Phase 1)

Item	Setting
Notebook	<code>Comparative_Study_CNN_refactored.ipynb</code>
Classes	36
Image size	224 × 224
Batch size	24
Epochs	10
Learning rate	1e-4
Optimizer	Adam
Loss	Sparse categorical crossentropy
Train/val split	80/20, seed 123
Models	MobileNetV2, ResNet50, VGG16, EfficientNetB0, DenseNet121
Demo	<code>leaf_disease_app.py</code>
Grad-CAM	<code>gradcam_saved_weights.py</code>
Results export	<code>comparison_results.csv</code>

Appendix B — Supported Classes

Crop model (36): Apple (4), Blueberry (1), Cherry (2), Corn (4), Grape (4), Orange (1), Peach (2), Pepper (2), Potato (3), Raspberry (1), Soybean (1), Squash (1), Strawberry (2), Tomato (8), Background (1)

Medicinal model (13): Camphor (3), HariTaki (3), Neem (4), Sojina (3)

Appendix C — One-Minute Oral Summary

We trained five ImageNet backbones with the same head and optimizer for 10 epochs on 36 plant-disease classes. ResNet50 achieved the best validation scores (~99% accuracy, ~0.99 MCC). EfficientNetB0 was second with similar training time to MobileNetV2 and was chosen for deployment in our AgroVision AI web app. We then built a full farmer assistance platform with AI guidance, multilingual support, weather risk, voice, WhatsApp, and progress tracking. Phase 2 will ensemble top models — likely ResNet50 + EfficientNetB0 — and evaluate on held-out field images.